

What I can prepare after you upload it

1. Git Lab Tasks 1–5

- Branch creation
- Feature development
- Multi-level merging
- Stashing and restoring changes
- Merge-conflict creation and resolution in VS Code
- `git diff` and `git restore`
- `git reset --soft`
- `git reset --hard`

2. Complete Git command sequence, for example:

```
3. git clone <repository-url>
4. git checkout -b dev
5. git checkout -b feature-...
6. git add .
7. git commit -m "... "
8. git push -u origin feature-...
9. git merge feature-...
10. git stash
11. git stash pop
12. git diff
13. git restore <file>
14. git reset --soft HEAD~1
    git reset --hard HEAD~1
```

15. Lab report

- Task description
- Step-by-step procedure
- Git commands
- Explanation of every important command
- Screenshots/places for screenshots
- Results and verification
- Final GitHub repository link

16. Correct filename

C243186_lab1.pdf

assuming your student ID is C243186.

17. GitHub repository

I can guide you through creating/pushing the complete repository. If you provide the repository URL or use your existing CSE GitHub repository, I can tailor every command to it.

Task 1 — Feature Branch Development and Multi-Level Merging

bash

1. Create dev branch from main

```
git checkout main
git pull origin main
git checkout -b dev
git push -u origin dev
```

2. Create three feature branches from dev

```
git checkout -b feature-login dev
git checkout -b feature-signup dev
git checkout -b feature-profile dev
```

3. On each feature branch: create/modify file, stage, commit, push

```
git checkout feature-login
echo "Login feature code" > login.txt
git add login.txt
git commit -m "Add login feature"
git push -u origin feature-login
```

```
git checkout feature-signup
echo "Signup feature code" > signup.txt
git add signup.txt
git commit -m "Add signup feature"
git push -u origin feature-signup
```

```
git checkout feature-profile
echo "Profile feature code" > profile.txt
git add profile.txt
git commit -m "Add profile feature"
git push -u origin feature-profile
```

4. Merge all three into dev

```
git checkout dev
git merge feature-login
git merge feature-signup
git merge feature-profile
```

5. Push dev

```
git push origin dev
```

```
# 6. Merge dev into main
```

```
git checkout main
```

```
git merge dev
```

```
git push origin main
```

Task 2 — Stash Changes

```
bash
```

```
git checkout -b feature-A dev
```

```
echo "work in progress" >> feature-a.txt
```

```
git stash push -m "WIP feature A"
```

```
git checkout -b feature-B dev
```

```
echo "different change" > feature-b.txt
```

```
git add feature-b.txt
```

```
git commit -m "Add feature B change"
```

```
git push -u origin feature-B
```

```
git checkout feature-A
```

```
git stash list
```

```
git stash pop
```

```
cat feature-a.txt    # verify restored
```

Task 3 — Merge Conflict Resolution

```
bash
```

```
git checkout dev
```

```
git checkout -b conflict-A
```

```
# edit shared.txt line 1 -> "Version A"
```

```
git add shared.txt && git commit -m "Change line to Version A"
```

```
git checkout dev
```

```
git checkout -b conflict-B
# edit shared.txt line 1 -> "Version B"
git add shared.txt && git commit -m "Change line to Version B"

git checkout dev
git merge conflict-A          # succeeds
git merge conflict-B          # CONFLICT

# Open shared.txt in VS Code -> resolve using the conflict UI
git add shared.txt
git commit -m "Resolve merge conflict in shared.txt"
git push origin dev
```

Screenshot the conflict markers (<<<<<<, =====, >>>>>>) and the VS Code "Accept Incoming/Current/Both" toolbar.

Task 4 — Restore Uncommitted Changes

bash

```
git checkout feature-A
echo "unwanted change" >> feature-a.txt
git diff
git restore feature-a.txt
git diff          # should be empty
```

Task 5 — Undo Commits Using Reset

bash

```
git checkout -b reset-demo dev
echo "first" > file.txt && git add . && git commit -m "First commit"
echo "second" >> file.txt && git add . && git commit -m "Second commit"
```

```
git reset --soft HEAD~1
git status          # changes staged
git commit -m "Second commit (re-committed)"
```

```
git reset --hard HEAD~1
git status          # clean
git log --oneline   # verify
```

Git Commands Cheat-Sheet (for your report)

Command	What it does
<code>git checkout -b <name></code>	Creates a new branch and switches to it
<code>git add <file></code>	Stages changes for commit
<code>git commit -m "msg"</code>	Records staged changes with a message
<code>git push -u origin <branch></code>	Pushes branch to remote and sets upstream
<code>git merge <branch></code>	Merges the specified branch into the current one
<code>git stash</code>	Temporarily saves uncommitted changes
<code>git stash pop</code>	Restores the most recent stash and removes it from the stash list
<code>git diff</code>	Shows unstaged changes
<code>git restore <file></code>	Discards uncommitted changes in a file

Command

What it does

```
git reset --soft HEAD~1
```

Removes last commit, keeps changes staged

```
git reset --hard HEAD~1
```

Removes last commit and discards changes

```
git log --oneline
```

Shows commit history in one line per commit